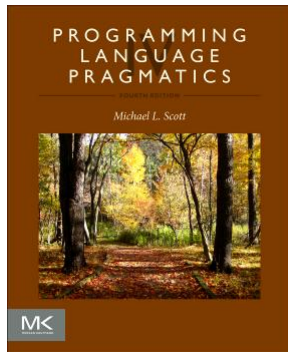


Chapter 8 :: Composite Types

Programming Language Pragmatics, Fourth Edition

Michael L. Scott



Symbol Table

- Symbol table stores information about all variables defined in the program
- The typical information kept is
 - symbol name
 - nature of the symbol (local variable, parameter, struct field etc.)
 - type of the symbol
 - scope info
 - address info (typically offset from some base address)
 - Etc.
- Various Implementations exist.
- List of Hash Tables or simply Hashing

Symbol Table Example

Index	Symbol	Type
1	void	type
2	int	type
3	getint	func: (1) $\rightarrow$ (2)
4	putint	func: (2) $\rightarrow$ (1)
5	i	(2)
6	j	(2)

```
int main() {  
    int i = getint(), j = getint();  
    while (i != j) {  
        if (i > j) i = i - j;  
        else j = j - i;  
    }  
    putint(i);  
}
```

Type Descriptor Table

- Type descriptor table stores information about all types defined in the program
- The typical information kept is
 - Name of type
 - Nature of type (struct, array, pointer etc.)
 - Name of every field and its address (in case of struct)
 - Type of object being pointed (in case of struct)
 - How many pointers are present in the type and their offsets
 - Etc.

Composite Types

- Nonscalar types are usually called composite, or constructed types.
- Created by applying a type constructor to one or more simpler types.
- Common composite types include records (structures), variant records (unions), arrays, sets, pointers, lists, and files.

Records (Structures)

- Records (structures) were introduced by Cobol, and have been supported by most languages since the 1960s.
- A record consists of collection of fields, each of which belongs to a different simpler type.

Records (Structures)

- Usually laid out contiguously
- Possible holes for alignment reasons
- Smart compilers may rearrange fields to minimize holes (C compilers promise not to)
- Implementation problems are caused by records containing dynamic arrays

Records (Structures)

C Example:

```
struct element {  
    char name[2];  
    int atomic_number;  
    double atomic_weight;  
    _Bool metallic;  
};
```

Pascal Example:

```
type two_chars = packed array  
    [1..2] of char;  
type element = packed record  
    name : two_chars;  
    atomic_number : integer;  
    atomic_weight : real;  
    metallic : Boolean  
end;
```

Records (Structures)

- Memory layout and its impact (structures)

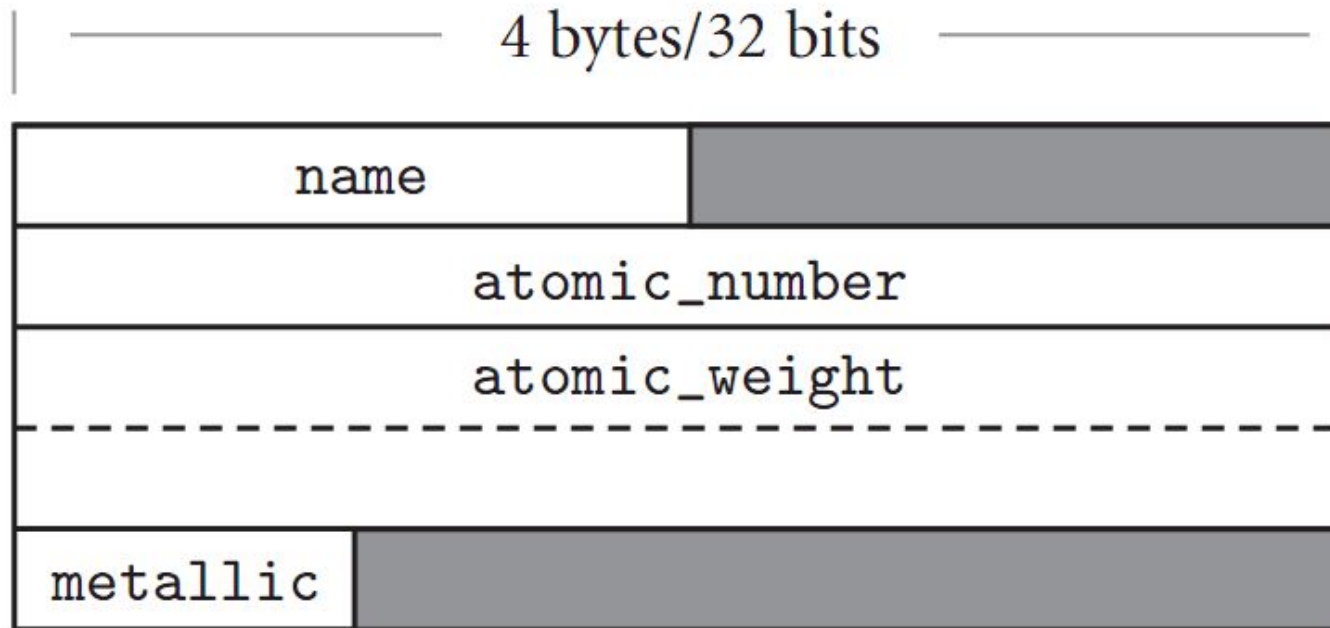


Figure 8.1 Likely layout in memory for objects of type `element` on a 32-bit machine. Alignment restrictions lead to the shaded “holes.”

Records (Structures)

- Memory layout and its impact (structures)

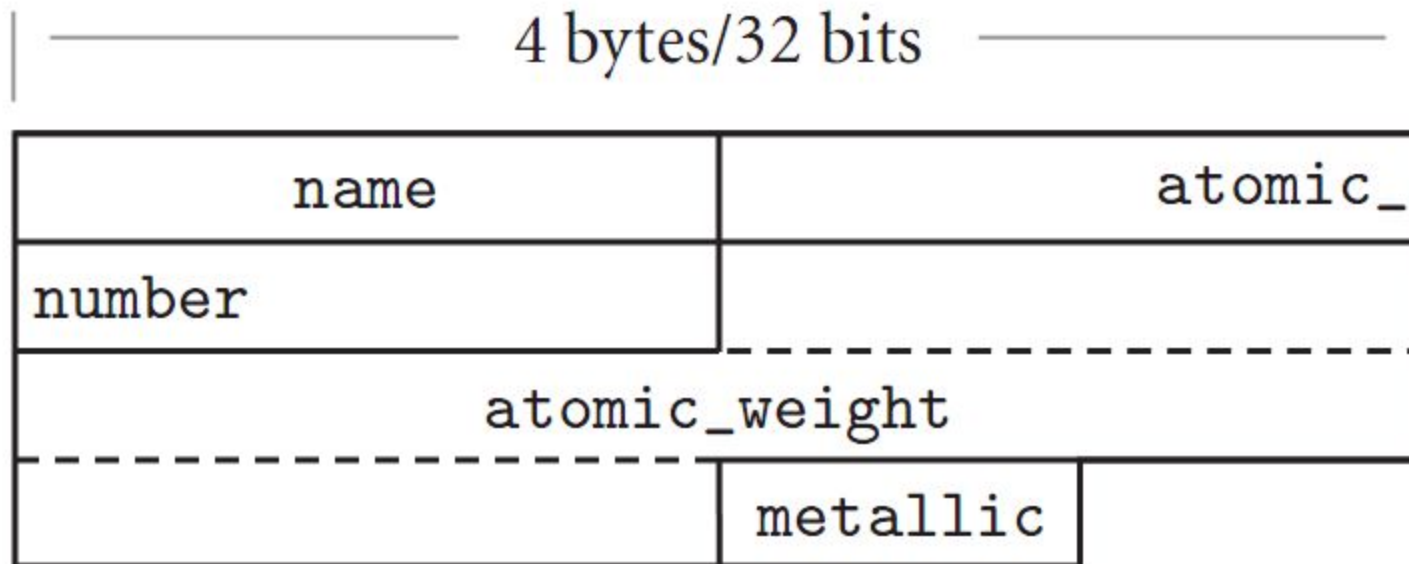


Figure 8.3 Likely memory layout for packed element records. The `atomic_number` and `atomic_weight` fields are nonaligned, and can only be read or written (on most machines) via multi-instruction sequences.

Records (Structures)

- Memory layout and its impact (structures)

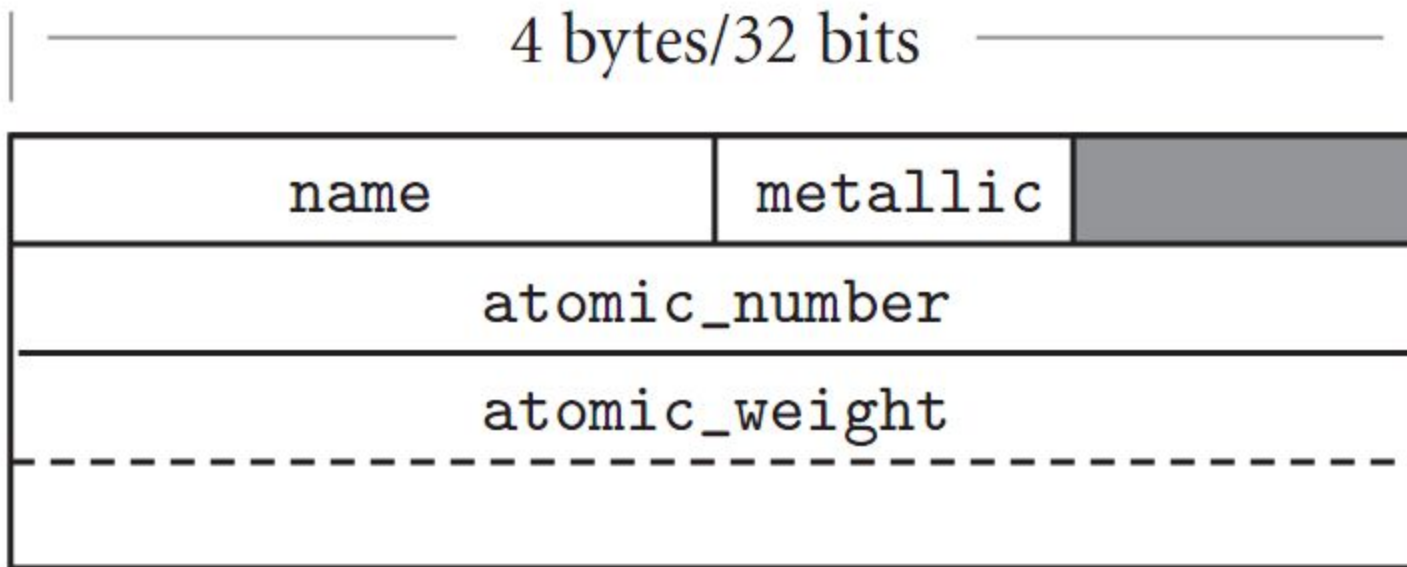


Figure 8.4 Rearranging record fields to minimize holes. By sorting fields according to the size of their alignment constraint, a compiler can minimize the space devoted to holes, while keeping the fields aligned.

Accessing a field of a record (struct)

- $x = S.f$
- Assume register $r1$ (it can be specific register like frame pointer in case a record is defined locally within the function) has starting address of struct S
- `MOV  $r2$ , offset of  $f$  in  $S$`
 - `/* obtained from symbol table`
`Additional code is required to fetch this */`
- `MOV ( $x$ ),  $r1(r2)$`
- Once the offset has been used (considering all statements in the program), this information is not required at run-time.

Whole Record (struct) operations

- Record Copy

- Code can be generated to copy every field one at a time
(Good for small sized records)

- Comparison of two records is also done field by field

- Code can be generated to copy blocks of memory areas
from source to destination (Good for large sized
records)

- Holes (padding) bytes can have default values (zero) to enable
comparison of two records

- Compiler generates code for customized field by field
comparison for comparing two records

Unions (Variant records)

```
union { int i; float f; double d;} u;
```

```
...
```

```
u.f = 3.0;
```

```
...
```

```
printf("%f", u.f);
```

- Space allotted to the union is the space required for the maximum sized field (in this case `sizeof(double)`, say 8 bytes).
- All fields of the union share the same space.
- The programmer has to keep track of the field type that has been assigned so the same field is accessed next.

Unions (Variant records)

- Unions (variant records)
 - overlay space
 - cause problems for type checking
- Lack of tag or discriminator means we don't know what is there (some languages, like Pascal, allow a tag that is maintained by the programmer)

Variant Record in Pascal

```
type element = record
  name : two_chars;
  atomic_number : integer;
  atomic_weight : real;
  metallic : Boolean;
  case naturally_occurring : Boolean of
    true : (
      source : string_ptr;
      prevalence : real;
    );
    false : (
      lifetime : real;
    )
  end;
end;
```

Records (Structures) and Variants (Unions)

- Memory layout and its impact (unions)

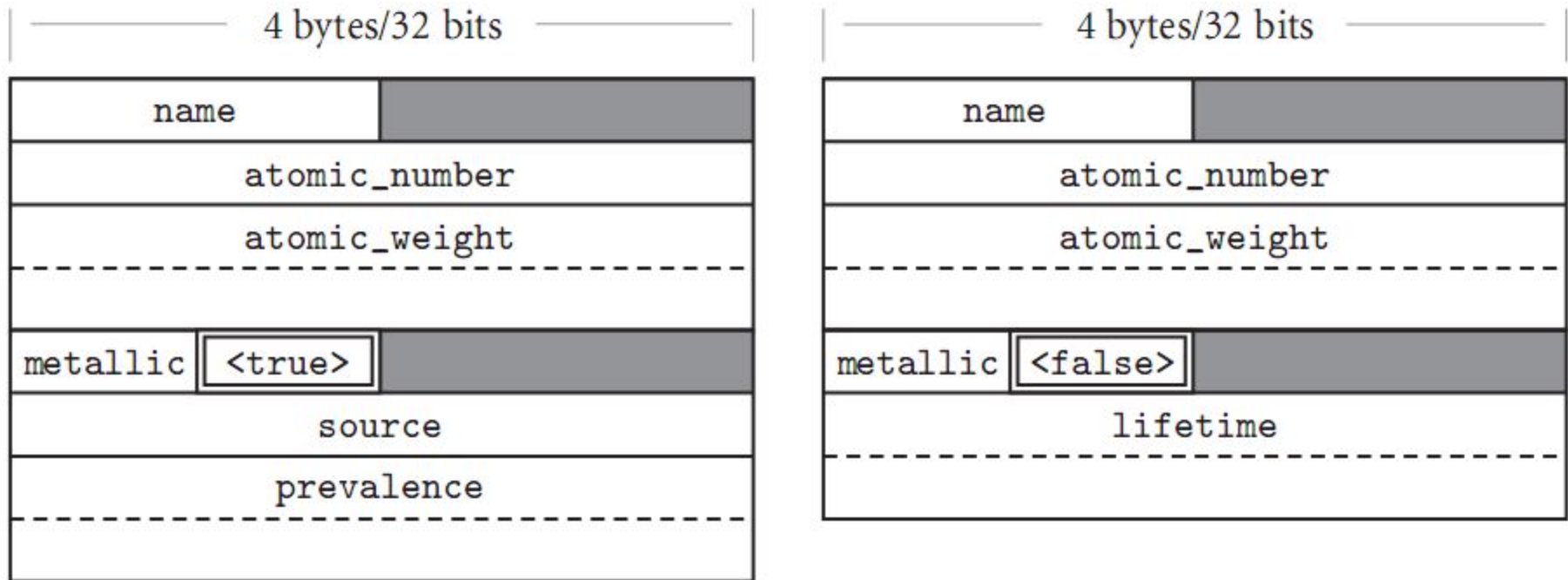


Figure 8.16 (CD) Likely memory layouts for element variants. The value of the naturally_ occurring field (shown here with a double border) is intended to indicate which of the interpretations of the remaining space is valid. Field source is assumed to point to a string that has been independently allocated.

Variants in Ada

```
type element (naturally_occurring : Boolean := true) is record
  name : string (1..2);
  atomic_number : integer;
  atomic_weight : real;
  metallic : Boolean;
  case naturally_occurring is
    when true =>
      source : string_ptr;
      prevalence : real;
    when false =>
      lifetime : real;
  end case;
end record;
```

Declaration of the discriminant of the record in its header is done, also specified a default value for it.

A declaration of a variable of type element has the option of accepting this default value:

```
copper : element;
```

or we can override it:

```
plutonium : element (false);
```

```
neptunium : element (naturally_occurring => false);
```

If the type declaration for element did not specify a default value for naturally_occurring, then all variables of type element would have to provide a value.

These rules guarantee that the tag field of a variant record is never uninitialized.

Arrays

- Arrays are the most common and important composite data types
- Unlike records, which group related fields of disparate types, arrays are usually homogeneous
- Semantically, they can be thought of as a mapping from an *index type* to a *component* or *element type*

Arrays

- Dimensions, Bounds, and Allocation
 - *global lifetime, static shape* — If the shape of an array is known at compile time, and if the array can exist throughout the execution of the program, then the compiler can allocate space for the array in static global memory
 - *local lifetime, static shape* — If the shape of the array is known at compile time, but the array should not exist throughout the execution of the program, then space can be allocated in the subroutine's stack frame at run time.
 - *local lifetime, shape bound at elaboration time*
 - *Heap??*

Arrays

```
-- Ada:  
procedure foo (size : integer) is  
  M : array (1..size, 1..size) of real;  
  ...  
begin  
  ...  
end foo;
```

```
// C99:  
void foo(int size) {  
    double M[size][size];  
    ...  
}
```

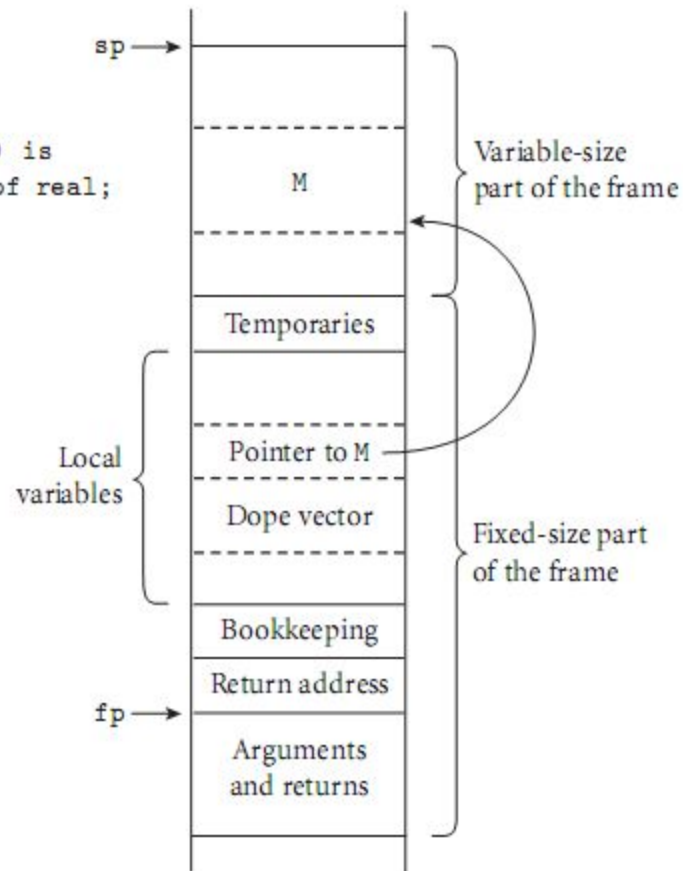


Figure 8.7 Elaboration-time allocation of arrays in Ada or C99.

Arrays

- Contiguous elements (see Figure 8.8)
 - column major - only in Fortran
 - row major
 - used by everybody else
 - makes array `[a..b, c..d]` the same as array `[a..b]` of array `[c..d]`

Arrays

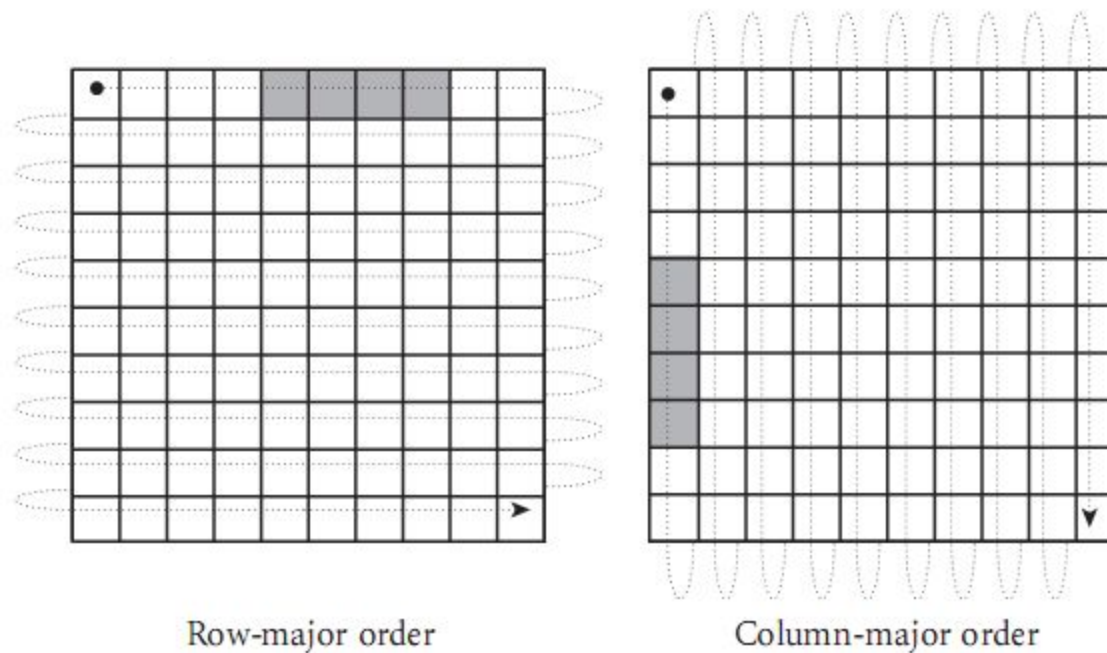


Figure 8.8 Row- and column-major memory layout for two-dimensional arrays. In row-major order, the elements of a row are contiguous in memory; in column-major order, the elements of a column are contiguous. The second cache line of each array is shaded, on the assumption that each element is an eight-byte floating-point number, that cache lines are 32 bytes long (a common size), and that the array begins at a cache line boundary. If the array is indexed from $A[0,0]$ to $A[9,9]$, then in the row-major case elements $A[0,4]$ through $A[0,7]$ share a cache line; in the column-major case elements $A[4,0]$ through $A[7,0]$ share a cache line.

Arrays

• Example 8.25 Indexing a contiguous array:

Suppose

A : array [L1..U1] of array [L2..U2] of array [L3..U3]
of element;

$$D1 = U1 - L1 + 1$$

$$D2 = U2 - L2 + 1$$

$$D3 = U3 - L3 + 1$$

Let

S3 = size of element

$$S2 = D3 * S3$$

$$S1 = D2 * S2$$

▫ Size of a row (S2) is the size of an individual element (S3) times the number of elements in a row (assuming the row-major order)

▫ Size of a plane (S1) is the size of row (S2) times the number of rows in a plane.

Arrays

$$\begin{aligned} A[i, j, k] = & \text{Address of } A + (i-L1) * S1 \\ & + (j-L2) * S2 \\ & + (k-L3) * S3 ; \end{aligned}$$

$$\begin{aligned} = & A - (L1 * S1 + L2 * S2 + L3 * S3) \\ & + (i * S1 + j * S2 + k * S3) ; \end{aligned}$$

Arrays

Ex.1: `int A[2][3][4] = {{0,1,2,3}, {1,2,3,4}, {2,3,4,5},
{3,4,5,6}, {4,5,6,7}, {5,6,7,8}};`

Calculate `A[1, 1, 1]`; Address of the element in the array

$$S3 = 4, S2 = (4-0) * S3 = 4 * 4 = 16$$

$$S1 = (3-0) * S2 = 3 * 16 = 48$$

$$A[1, 1, 1] = A + (1-0) * 48 + (1-0) * 16 + (1-0) * 4 \\ = A + 48 + 16 + 4 = \mathbf{A + 68}$$

Ex.2: `int A[3][4][5] (s3=4, s2=20, s1=80)`

`{{0,1,2,3,4}, {0,1,2,3,4}, {0,2,3,4,5}, {1,3,4,5,6},
{0,1,2,3,4}, {0,2,3,4,5}, {1,3,4,5,6}, {2,4,5,6,7}
{0,1,2,3,4}, {0,1,2,3,4}, {0,2,3,4,5}, {1,3,4,5,6}}`

Calculate `A[2, 2, 4]` = $A + (2-0) * 80 + (2-0) * 20 + (4-0) * 4$
 $= \mathbf{A + 216}$

Arrays

- **Example 8.25** (continued)

We could compute all that at run time, but we can make do with fewer subtractions:

$$\begin{aligned} &== \text{address of } A + \\ &\quad (i * S1) + (j * S2) + (k * S3) \\ &\quad - [(L1 * S1) + (L2 * S2) + (L3 * S3)] \end{aligned}$$

The stuff in square brackets is compile-time constant that depends only on the type of A.

The constant is kept in Symbol table entry for A.

Accessing an element of an contiguous array

$x = A[i, j, k];$

Assume address of A is in r1

SUB r1, compile time constant expression from symbol table entry of A

MOV r2, i

MUL r2, S1 /* S1 kept in symbol table entry of A */

ADD r1, r2

MOV r2, j

MUL r2, S2 /* S2 kept in symbol table entry of A */

ADD r1, r2

MOV r2, k

MUL r2, S3 /* S3 kept in symbol table entry of A */

ADD r1, r2

MOV x, (r1)

Arrays

- Two layout strategies for arrays (Figure 8.9):
 - Contiguous elements (good locality of reference)
 - Row pointers (poor locality of reference)
- Row pointers
 - an option in C
 - allows rows to be put anywhere - nice for big arrays on machines with segmentation problems
 - nice for matrices whose rows are of different lengths
 - e.g. an array of strings
 - requires extra space for the pointers

Arrays

```
char days[][10] = {
    "Sunday", "Monday", "Tuesday",
    "Wednesday", "Thursday",
    "Friday", "Saturday"
};
...
days[2][3] == 's'; /* in Tuesday */
```

S	u	n	d	a	y	/			
M	o	n	d	a	y	/			
T	u	e	s	d	a	y	/		
W	e	d	n	e	s	d	a	y	/
T	h	u	r	s	d	a	y	/	
F	r	i	d	a	y	/			
S	a	t	u	r	d	a	y	/	

```
char *days[] = {
    "Sunday", "Monday", "Tuesday",
    "Wednesday", "Thursday",
    "Friday", "Saturday"
};
...
days[2][3] == 's'; /* in Tuesday */
```

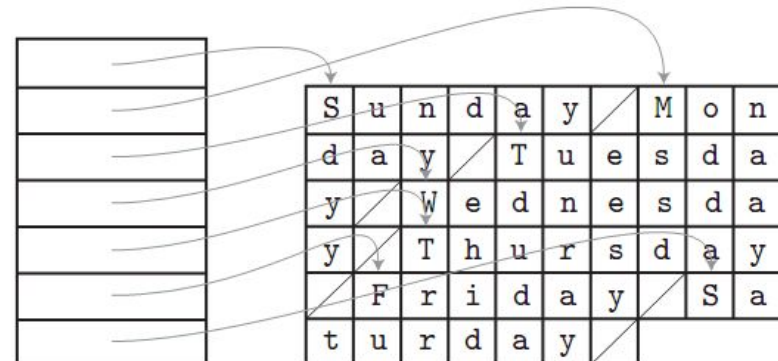


Figure 8.9 Contiguous array allocation v. row pointers in C. The declaration on the left is a true two-dimensional array. The slashed boxes are NUL bytes; the shaded areas are holes. The declaration on the right is a ragged array of pointers to arrays of characters. In both cases, we have omitted bounds in the declaration that can be deduced from the size of the initializer (aggregate). Both data structures permit individual characters to be accessed using double subscripts, but the memory layout (and corresponding address arithmetic) is quite different.

Accessing an element of an array – row pointer layout

$x = A[i, j, k];$

Assume address of A is in r1

MOV r2, i

MUL r2, #4 /*size of a pointer*/

MOV r1, r1(r2)

MOV r2, j

MUL r2, #4 /*size of a pointer*/

MOV r1, r1(r2)

MOV r2, k

MUL r2, #8

/*size of an element of A from symbol table entry (say 8 bytes) */

MOV r1, r1(r2)

MOV x, (r1)

Strings

- In some languages, strings are really just arrays of characters
- In others, they are often special-cased, to give them flexibility (like polymorphism or dynamic sizing) that is not available for arrays in general
 - It's easier to provide these things for strings than for arrays in general because strings are one-dimensional and (more importantly) non-circular

Sets

- A programming language set is an unordered collection of an arbitrary number of distinct values of a common type.
- Available in many programming languages, e.g. Pascal, Icon, Python, etc.
- Pascal** supports sets of any discrete type
- TYPE set-name = SET OF base-type;
- TYPE CharSet = SET OF CHAR; {a set of characters}
- TYPE DigitSet = SET OF 0..9; { a set of digits }
- TYPE WeekDayType = (Sun, Mon, Tue, Wed, Thur, Fri, Sat);
- WorkDays SET OF WeekDayType;
- WorkDays := [Mon, Tue, Wed, Thur, Fri];

Sets

Operations:

Membership -

Using IN operator-returns a BOOLEAN value.

Basic Operations

$A := B + C$; (* union; $A := \{x \mid x \text{ is in } B \text{ or } x \text{ is in } C\}$ *)

$A := B * C$; (* intersection; $A := \{x \mid x \text{ is in } B \text{ and } x \text{ is in } C\}$ *)

$A := B - C$; (* difference; $A := \{x \mid x \text{ is in } B \text{ and } x \text{ is not in } C\}$ *)

-**Icon** supports sets of characters (called *csets*), but not sets of any other base type.

-**Python** supports sets of arbitrary type;

- Sets appear in the standard libraries of many object-oriented languages, including C++, Java, and C#.

Sets

- Many possible implementations
 - Characteristic array or Bitsets : it employs a bit vector whose length (in bits) is the number of distinct values of the base type.
 - A one in the k^{th} position in the bit vector indicates that the k^{th} element of the base type is a member of the set; a zero indicates that it is not.
 - In a language that uses ASCII, a set of characters would occupy 128 bits-16 bytes.
 - Things like intersection, union, membership, etc. can be implemented efficiently with bitwise logical instructions
 - Hash tables can also be used to implement sets
 - Some languages place limits on the sizes of sets to make it easier for the implementor

Pointers And Recursive Types

- A recursive type is one whose objects may contain one or more references to other objects of same type.
- In languages that use a value model of variables, recursive types require the notion of a pointer: a variable (or field) whose value is a reference to some object.

```
struct foo {struct foo * x;}           struct foo {int data, struct foo *x;} 
```

- Pointers serve two purposes:
 - Efficient access to elaborated objects (as in C)
 - dynamic creation of linked data structures, in conjunction with a heap storage manager
- Several languages (e.g. Pascal, Ada 83) restrict pointers to accessing things in the heap
- Pointers are used with a value model of variables
 - They aren't needed with a reference model

Pointers And Recursive Types

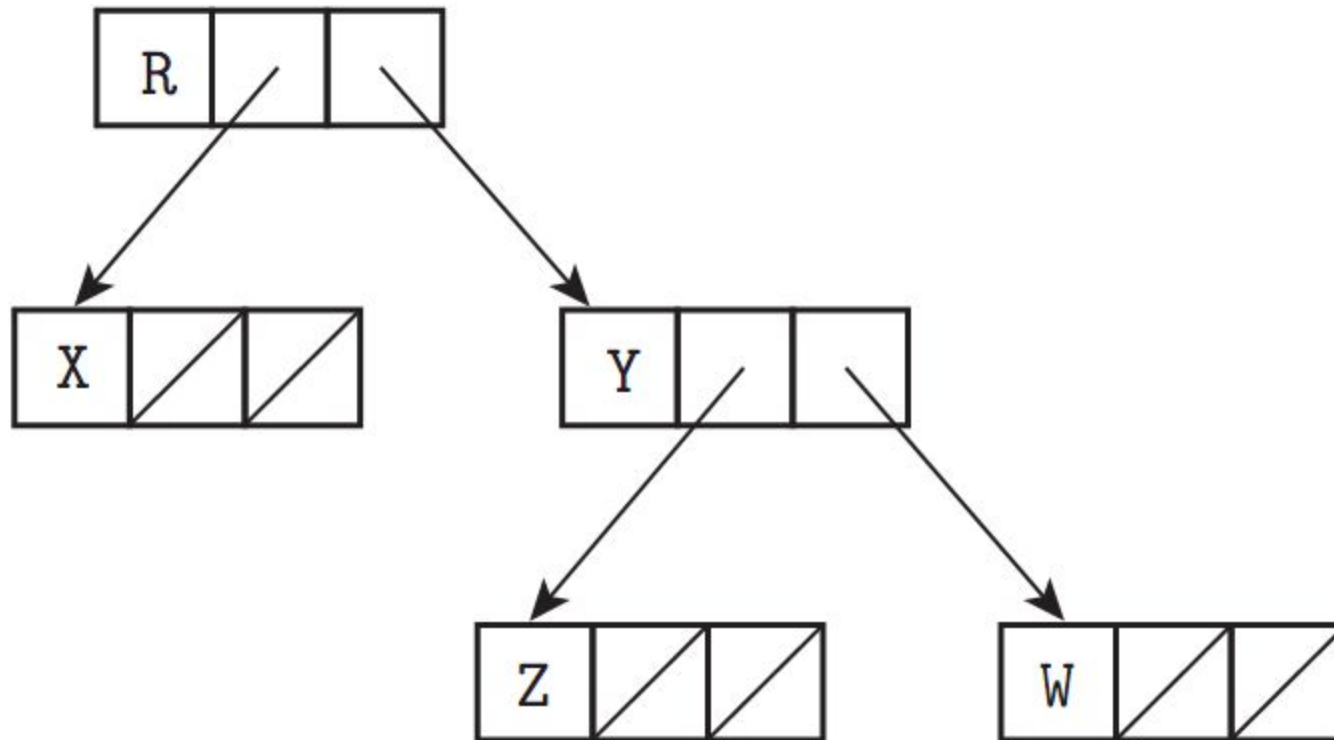


Figure 8.13 Typical implementation of a tree in a language with explicit pointers. As in Figure 8.12, a diagonal slash through a box indicates a null pointer.

Pointers And Recursive Types

- A pointer is a high-level concept: a reference to an object.
- An address is a low-level concept: the location of a word in memory.
- Pointers are often implemented as addresses.
- On a machine with a segmented memory architecture, a pointer may consist of a segment id and an offset within the segment.

Pointers

```
int * Pi;  
char *Pc;  
int i;  
char c;  
Pc = &c;  
Pi = &i;  
char c1 = *Pc;  
int i1 = *Pi;
```

```
void * ??
```

Generic pointer

Pointer	Contents
(P _c) 1000-1001	1012
(P _i) 1004-1005	1008
1008-1011	int i
1012-1012	char c
1013-1015	??
1016-1016	c1
1017-1019	??
1020-1023	i1

Pointers And Recursive Types

- C pointers and arrays

```
int *a == int a[]
```

```
int **a == int *a[]
```

```
int **a, int *a[]    pointer to pointer to int
```

```
int *a[n], n-element array of row pointers
```

```
int a[n][m], 2-d array
```

Pointers And Recursive Types

- Compiler has to be able to tell the size of the things to which you point
 - So the following aren't valid:

```
int a[][]      bad  
int (*a)[]     bad
```
 - C declaration rule: read right as far as you can (subject to parentheses), then left, then out a level and repeat

```
int *a[n], n-element array of pointers to integer  
int (*a)[n], pointer to n-element array of  
integers
```

Pointers And Recursive Types

- **`int * p[10];`**
 - `// Go to right of p, and come back on left side`
(Array of pointers each pointing to an integer).
- **`int (*p) [10];`**
 - `// Go to right of p, nothing is there, then`
`goto left of p and conclude it is a`
pointer(pointer to an array of 10 integers)
- **`int * fun (float, char);`**
 - function returning a pointer to integer.
- **`int (* fun) (float, char);`**
 - `fun` is a pointer which points to a function taking `float` and `char` as parameters and returns an integer.

Pointers And Recursive Types

- If `a` is an array, `sizeof(a) / sizeof(a[0])` returns the number of elements in the array.
- If pointers occupy 4 bytes and double-precision floating-point numbers occupy 8 bytes, then given
- `double *a; /* pointer to double */`
- `double(*b)[10]; /*pointer to array of 10 doubles */`
- `sizeof(a) = sizeof(b) = 4,`
- `sizeof(*a) = sizeof(*b[0]) = 8, and`
- `sizeof(*b) = 80.`
- `void f(int len) {`
- `int A[len]; /* sizeof(A) == len * sizeof(int) */`
- **Size of array??**

Pointers And Recursive Types

- **Operations on Pointers**
- Allocation and deallocation of objects in the heap
 - Dereferencing of pointers to access the objects to which they point
 - Assignment of one pointer into another.
- Depends on Value Model or Reference Model
 - Functional Languages - Reference model
 - Imperative Languages- Mixed
 - In C, Pascal, or Ada, (value model)
 - $A := B$ puts the value of B into A.
 - If we want B to refer to an object, and we want $A := B$ to make A refer to the object to which B refers, then A and B must be pointers
 - In Clu and Smalltalk (reference model)
 - $A := B$ always makes A refer to the same object to which B refers.

Pointers And Recursive Types

- Java
- Variables of built-in Java types integers, floating-point numbers, characters, and Booleans employ a value model;
- $A := B$ in Java places the value of B into A if A and B are of built-in type;
- Variables of user-defined types (strings, arrays, and other objects in the object-oriented sense of the word) employ a reference model.
- It makes A refer to the object to which B refers if A and B are of user-defined type.

Dangling References

- Something that should have been deleted, but we do not delete it.
- Node * P1, * P2;

```
If( P1 = (Node *) malloc (sizeof(Node)) ==NULL)
    exit(1)
```

```
1. p1-> data =10
   p1-> next = NULL;
2. p2 =p1;
3. free(p1);
```

1. P1

2. P2



3. P1 deleted

3. P2 exists but points to whom?

3. P2 becomes the dangling reference

Memory Leak

- Access to a memory location that is deleted.
- Node * P1, * P2;

```
If( P1 = (Node *) malloc (sizeof(Node)) ==NULL)
```

```
exit(1)
```

```
If( P2 = (Node *) malloc (sizeof(Node)) ==NULL)
```

```
exit(1)
```

1. p1-> data =10

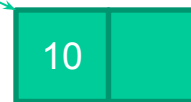
p1-> next = NULL;

2. p2-> data =20;

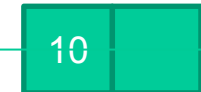
p2-> next = NULL;

3. p2 =p1;

1. P1

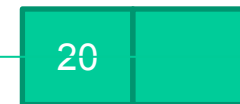


3. P1



3. P2

2. P2



3. Garbage

Pointers And Recursive Types

- Languages, e.g. C,C++,Pascal, and Modula-2, require the programmer to reclaim space explicitly.
- Modula-3, Java, C#, and all the functional and scripting languages, require the language implementation to reclaim unused objects automatically.
- **Explicit storage reclamation**
 - Simplifies the language implementation
 - Possibility that the programmer will forget to reclaim objects that are no longer live (Memory Leak), or
 - Will accidentally reclaim objects that are still in use (Dangling References).
- **Automatic storage reclamation (Garbage Collection) -**
 - Simplifies the programmer's task, but imposes certain run-time costs
 - How the language implementation is to distinguish garbage from active objects?

Pointers And Recursive Types

- Problems with dangling pointers are due to
 - explicit deallocation of heap objects
 - only in languages that *have* explicit deallocation
 - implicit deallocation of elaborated objects
- Two implementation mechanisms to catch dangling pointers
 - Tombstones
 - Locks and Keys

Tombstones

- Allow a language implementation can catch all dangling references to objects in both stack and heap.
- An extra level of indirection
- A pointer holds an address of the tombstone and the tombstone contains the address of the object.
- When an object is reclaimed, the tombstone is modified to contain a value (zero) that can not be a valid address.

Pointers And Recursive Types

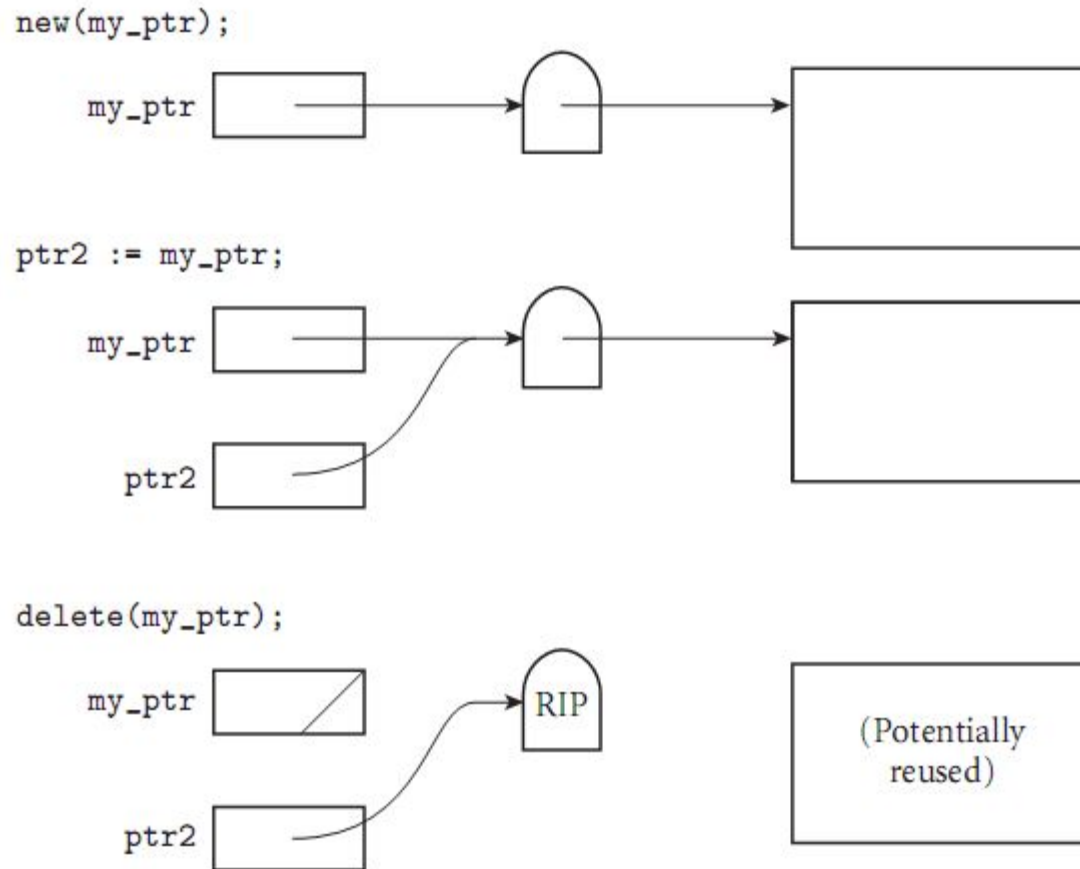


Figure 8.17 (CD) Tombstones. A valid pointer refers to a tombstone that in turn refers to an object. A dangling reference refers to an "expired" tombstone.

Tombstones

- Where are tombstones allocated from?
 - May be allocated from the heap
 - From a separate pool.
- Tombstones can be expensive
 - In time
 - In space

Tombstones

- **Time Overhead**
- Creation of tombstones when allocating heap objects
- Checking for validity on every access
- Most of the machines - validity can be free, CPU registers used to check an access is within bounds or not or else if goes beyond the area of the program, then the hardware interrupt occurs
- Double indirection

Tombstones

- **Space Overhead**
- Should we deallocate them or keep them as it is??
- Valuable side effect of Tombstones
 - Easy to change the location of an object in the heap.
 - Need not locate every pointer that refers to the object, but change the address in the tombstone.
- Storage Compaction
 - Keep all the blocks (dynamically allocated) together at one end of the heap in order to eliminate the external fragmentation.
- Usage:
 - Machintosh OS (Ver. 9 and above) used them internally for references to system objects such as “file and window” descriptors.

Lock and Keys

- An alternative to tombstones.
- Work with the objects in the heap.
- Every pointer is a tuple containing an address and a key.
- Object contains the lock.
- A pointer to an object is valid only if the key in pointer matches with the lock in the object.
- New object to be allocated on the heap comes with a new key value.
- keys and locks are generally are some serial numbers other than 0 or 1.
- When an object is reclaimed, its lock is changed to so arbitrary value (e.g., *zero*) so that the keys in any remaining pointers will not match.

Pointers And Recursive Types

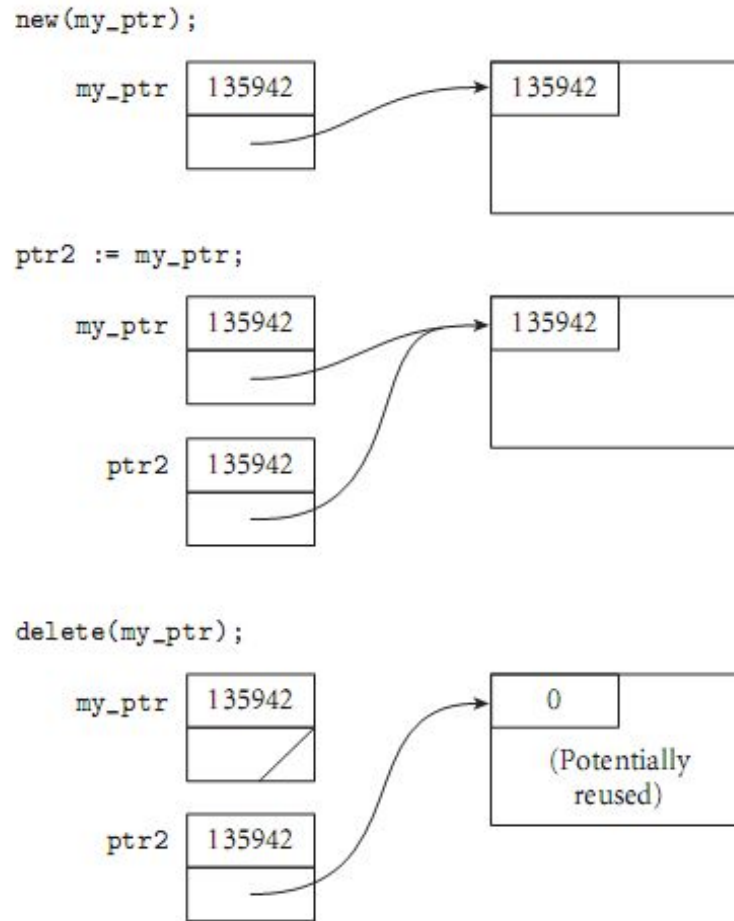


Figure 8.18 (CD) Locks and Keys. A valid pointer contains a key that matches the lock on an object in the heap. A dangling reference is unlikely to match.

Lock and Keys

- Significant overhead
- Extra word of storage to every pointer and every block in the heap.
- Increase the cost of copying one pointer into another.
- Cost of comparison between key and lock on every access.

Pointers And Recursive Types

- Problems with garbage collection
 - many languages leave it up to the programmer to design without garbage creation - this is VERY hard
 - others arrange for automatic garbage collection
 - reference counting
 - does not work for circular structures
 - works great for strings
 - should also work to collect unneeded tombstones

Pointers And Recursive Types

- Mark-and-sweep
 - commonplace in Lisp dialects
 - complicated in languages with rich type structure, but possible if language is strongly typed
 - achieved successfully in Java, C#, Scala, Go
 - complete solution impossible in languages that are not strongly typed
 - conservative approximation possible in almost any language (Xerox Portable Common Runtime approach)

Pointers And Recursive Types

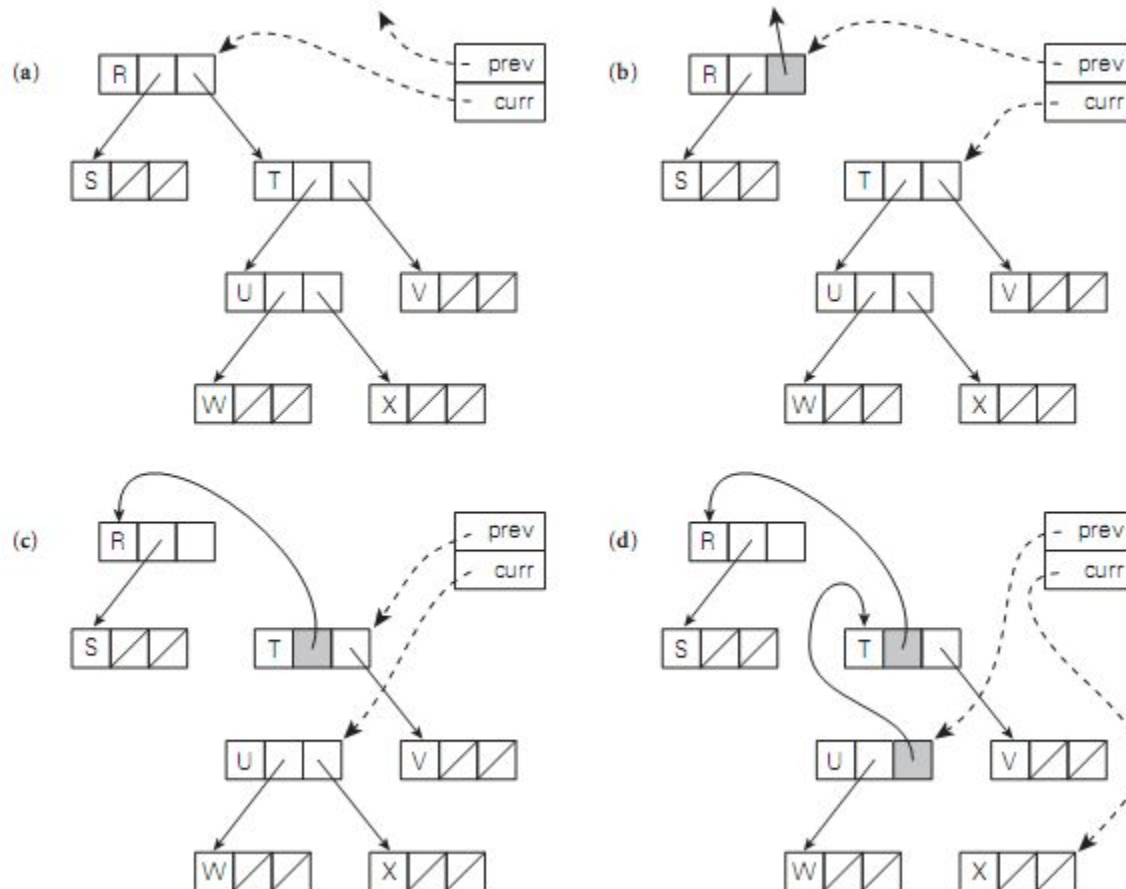


Figure 8.15 Heap exploration via pointer reversal.